

Implement Client Script & UI Policy (Incident)

Phase 1 - Brainstorming & Ideation Phase

Absolutely. For a **ServiceNow Incident** form, here's a brainstorming/ideation framework for implementing **Client Scripts** + **UI Policies**.

1. Start with the desired Incident behavior

Potential requirements to explore:

- **Make fields mandatory**
 - Make *Category* mandatory when creating an Incident.
 - Make *Subcategory* mandatory when a Category is selected.
 - Make *Close notes* mandatory when resolving/closing an Incident.
- **Show/hide fields**
 - Show *Assignment group* or additional troubleshooting fields only for certain Categories.
 - Show *Resolution code* and *Resolution notes* only when the Incident moves toward closure.
 - Hide technical fields from users who don't need them.
- **Make fields read-only**
 - Make certain fields read-only after the Incident is closed.
 - Prevent users from manually changing system-controlled fields.
- **Dynamic field behavior**
 - Automatically populate values based on another field.
 - Filter available Subcategories based on Category.
 - Change the available Assignment Groups depending on Category/Service.
- **Validation**
 - Prevent resolving an Incident without required resolution information.
 - Prevent invalid combinations of fields.
 - Display an informative message when user input doesn't satisfy a business rule.

2. Decide: Client Script or UI Policy?

A useful design rule:

Requirement	Prefer
Make a field mandatory	UI Policy
Make a field read-only	UI Policy
Show/hide a field	UI Policy
Complex conditional logic	Client Script
Validate user input	Client Script
React to a field changing	onChange Client Script

Requirement	Prefer
Set values when form loads	onLoad Client Script
Validate before submitting	onSubmit Client Script

In practice, **use a UI Policy when declarative behavior is sufficient**, and reserve Client Scripts for logic that needs JavaScript.

3. Example Incident scenario

Suppose the requirement is:

When an Incident's State changes to **Resolved**, Resolution code and Resolution notes should become mandatory.

UI Policy idea

Table: Incident [incident]

Condition:

State is Resolved

UI Policy Actions:

- Resolution code → Mandatory = True
- Resolution notes → Mandatory = True
- Resolution code → Visible = True
- Resolution notes → Visible = True

This is a good UI Policy candidate because there is no need for custom JavaScript.

4. Client Script ideas

onChange — Category

When Category changes:

```
Category = Hardware
↓
Show hardware-related fields
↓
Possibly set/filter Subcategory
```

For example:

```
function onChange(control, oldValue, newValue, isLoading) {
```

```

if (isLoading) {
    return;
}

if (newValue === 'hardware') {
    g_form.setVisible('subcategory', true);
}
}

```

The exact choice values should come from your instance rather than being assumed.

onSubmit — Resolution validation

Conceptually:

```

function onSubmit() {
    if (g_form.getValue('state') === '6' &&
        !g_form.getValue('close_notes')) {

        g_form.addErrorMessage(
            'Please provide resolution notes before resolving the Incident.'
        );

        return false;
    }

    return true;
}

```

Again, the actual State value should be verified in the target instance.

5. A good implementation exercise

You could build this incrementally:

Activity 1 — Basic UI Policy

Requirement:

When State = Resolved:

- Resolution code → mandatory
- Resolution notes → mandatory

Activity 2 — onChange Client Script

Requirement:

When Category changes:

- Dynamically alter another Incident field.
- Display an appropriate informational message.

Activity 3 — onLoad Client Script

Requirement:

When an existing Incident is opened:

- Detect its current state.
- Apply any additional client-side behavior required by the specification.

Activity 4 — onSubmit Client Script

Requirement:

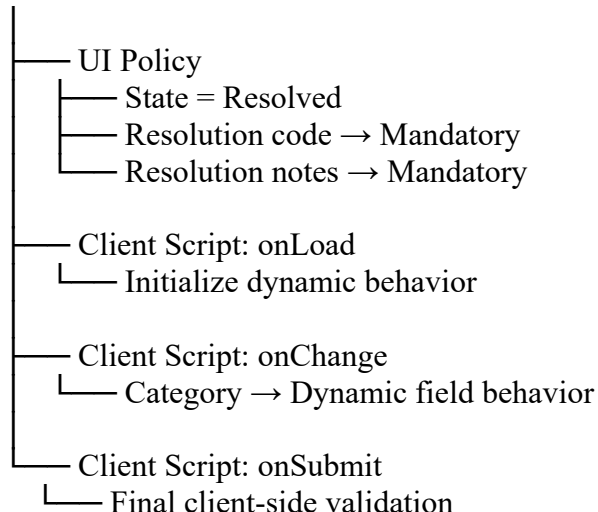
Before submission:

- Validate the Incident's data.
- Block submission if a required business condition isn't satisfied.

Activity 5 — Combine them

The final design could look like:

Incident Form



6. Important design considerations

During implementation, I'd specifically brainstorm around:

- **Avoid duplicating logic** between UI Policies and Client Scripts.
- Prefer **UI Policies for simple field state changes**.
- Keep Client Scripts focused and small.
- Consider whether the logic needs to work in **Workspace**, not just the classic form.
- Don't rely on Client Scripts for **security or server-side enforcement**; client-side controls can be bypassed.
- Test **new records, existing records, state changes, and form reloads**.
- Check behavior when a field changes **back and forth**, not just in one direction.
- Account for users who don't have the same roles or access to the fields.

If this is for a **ServiceNow hands-on task/interview**, we can next turn this brainstorming into a **complete Incident Client Script + UI Policy implementation exercise**, including the exact requirements, configuration steps, scripts, and test cases.